



TECHNISCHE
UNIVERSITÄT
WIEN

Introduction to Unix/Linux

191.002 Operating Systems VU
2026W

Luca Di Stefano, Jaroslav Klapalek, Joel Klimont, Felix Resch,
Peter Puschner
2026-10-06

- What is an Operating System?
 - UNIX, Linux, ...?
 - Why C?
-
- Using the shell
 - Process management
 - File system

The operating system as ...

1. An extended machine

- Provide simpler and easier to use abstractions of the underlying hardware
- Provide services that programs can obtain by a special interface

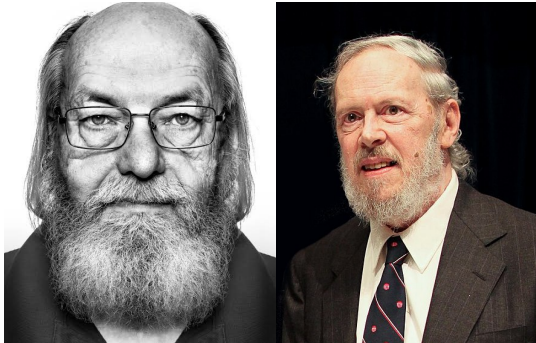
2. A resource manager

- Multiplexing/sharing resources in time and in space
- Create the illusion that a program has exclusive access to the resources

Important mechanisms: Processes, virtual memory, file system, ...

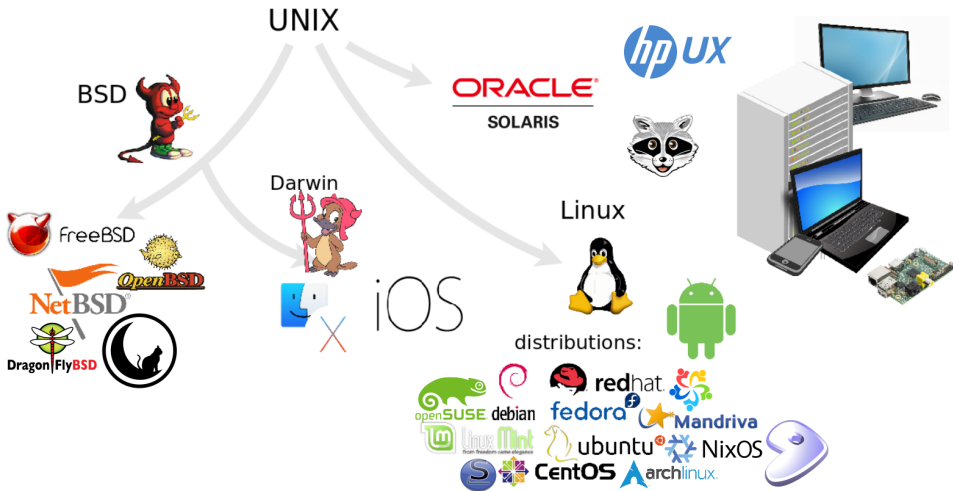
Origins

- 1969: Ken Thompson, Dennis Ritchie, et al.
- Initially Bell Labs, then spread to universities and companies

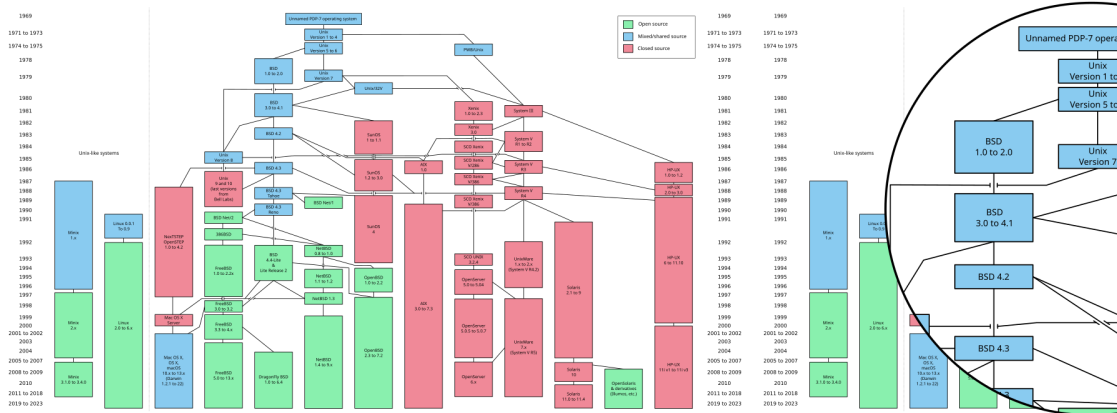


Source: Wikipedia

Unix-family OS everywhere

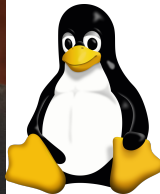


Evolution of Unix and Unix-like systems



Source: Wikipedia

Richard Stallman: GNU (1983) - Compiler, shell, tools for UNIX systems
Linus Torvalds: Linux (1991) - Free UNIX clone for PCs, using GNU tools¹



Source: Wikipedia

¹ <https://web.archive.org/web/20130509134305/http://groups.google.com/group/comp.os.minix/msg/b813d52cbc5a044b>

Distributions (distros)

- Kernel + Core utilities
- Pre-packaged tools
- Graphical shell (?)
- Package management tools
- Target specific platforms or use case
- (Specific CPUs, Docker, Embedded. . .)



Source: Wikipedia

... and many more

- L4 μ -Kernel Family (Jochen Liedtke)^a
- Plan 9 from Bell Labs^b
- Haiku^c



HAIKU



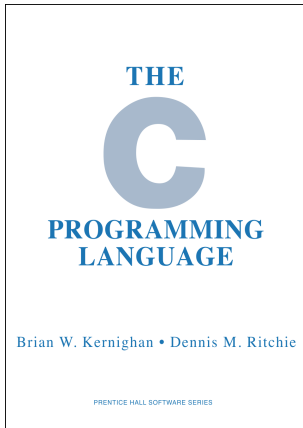
^a<https://os.inf.tu-dresden.de/L4/>

^b<https://9p.io/plan9/>

^c<https://www.haiku-os.org/>

History

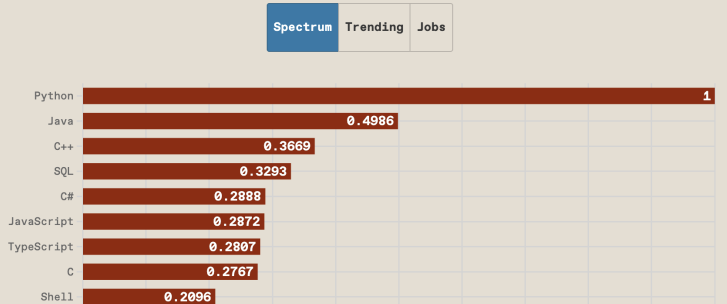
- Early adoption: 1972 (UNIX ported to C)
- 1978: “The C Programming Language” by (Kernighan & Ritchie) (“K&R C”)
- Tools improved over time
 - Compilers (cc, pcc, gcc, clang, ...)
 - Debuggers (gdb, lldb, ...)
 - Profilers (gprof, ...)
- Language evolved (ANSI C, C99, C11, C18, C23)
- Has not lost popularity and importance!



Source: Wikipedia

Top Programming Languages 2025

Click a button to see a differently weighted ranking



Source: IEEE Spectrum Rating of programming languages, 2025
<https://spectrum.ieee.org/top-programming-languages-2025>

Year	C	Java	Rust	1 st Place
2014	2 nd	1 st	-	Java
2016	1 st	2 nd	-	C
2022	2 nd	5 th	20 th	Python
2023	4 th	2 nd	18 th	Python
2024	9 th	2 nd	11 th	Python
2025	8 th	2 nd	14 th	Python

Simple and effective language

- Constructs clearly map to machine instructions
- Easy to implement **compilers**
- Modern compilers produce **very efficient code** for a vast number of **platforms**

Has influenced many other languages (C++, C#, Java, ...)

UNIX interface (system calls) designed for C

- Essentially no abstractions: full view of what is going on
- For instance, manual memory management = raw (virtual) memory addresses

High-level languages (Python, Ruby, ...):

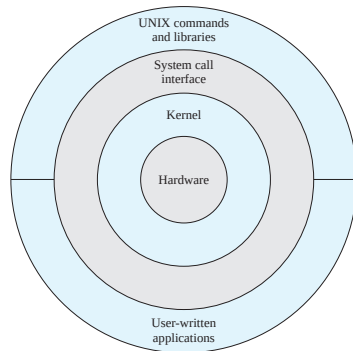
- Make performance trade-offs
- Are often **implemented in C**
- E.g., (C)Python's success is also (arguably) because of **C interfaces** (ctypes)²

Modern systems programming languages (Rust, Go, Zig, ...):

- Show promise (e.g., Rust in Linux kernel)
- After learning C you should understand **why** they make certain choices
- They **may** threaten C's **dominance** in the long term
- But C remains **relevant** for the foreseeable future

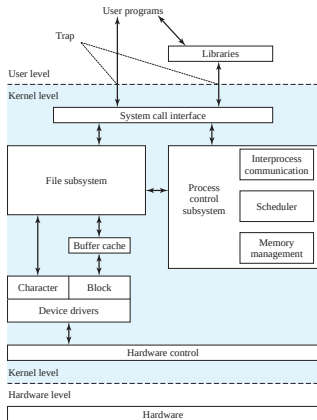
²<https://docs.python.org/3/library/ctypes.html>

- User space vs Kernel space
 - Kernel routines, device drivers, etc. run in privileged mode (*kernel mode*)
 - User applications request kernel services with *system calls*
- Multi-process and multi-user OS
 - Run more than one program concurrently
 - Users share resources
- Requires authentication (login)
 - Your permissions determine what you (= your applications) can do
 - Root user can do **everything**

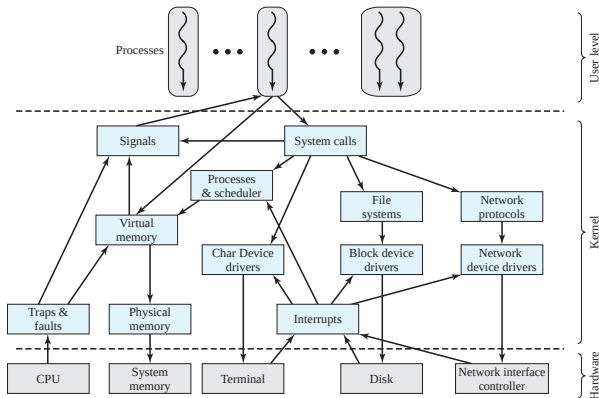


General Unix Architecture
(Source: W. Stallings, "Operating Systems. Internals and Design Principles")

³Some Unixes are microkernel-based (e.g., Minix).



Traditional UNIX Kernel



Linux Kernel Components

(Source: W. Stallings, "Operating Systems. Internals and Design Principles")

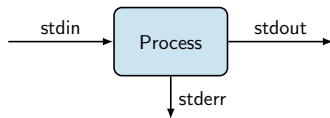
- A user-space program that
 - Interactively reads and interprets user input (**commands**) from a **prompt**
 - Starts other user programs
 - Executes **shell scripts**
- Examples: `sh`, `bash`, `zsh`, ...
- Typically started automatically when you login
- Typical shell prompt: `user@host:~$`
 - `user@host`: your user name and the computer's name
 - `~`: current working directory (home directory)
 - `$`: normal user (root user: `#`)
- Most shells also provide **completion** with `<tab>` (e.g., file)

- A **process** is how the OS represents a **running program**.
 - program code
 - program data (variables, ...)
 - context (state, program counter, processor registers, ...)
- A Unix system executes many processes **concurrently**
- Each process has an **identifier** (PID)
- Each process **except PID 1 (init)** has a **parent** that launched it
 - For instance, processes that you launch from a shell are **children** of the shell
- When they terminate, processes have an **exit status**
 - Aka exit value, return code, exit code ...
 - 0 = **success**
 - Other integers = **failure**
- `ps` – display current processes
- `pstree` – display process hierarchy (parent/children)

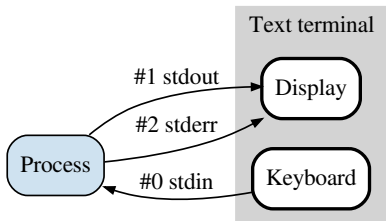
In UNIX “everything is a file” (a sequence of bytes). Processes use **file descriptors** to keep track of open files. **Not all files are stored on disk!**

Each process starts with three special file descriptors:

- Standard input (stdin, fd 0)
- Standard output (stdout, fd 1)
- Standard error (stderr, fd 2)



When started in a shell, the standard I/O is connected to the terminal.



```
$ echo Hi there
```

```
Hi there
```

```
$ echo "Hi there"
```

```
Hi there
```

```
$ echo "Hi           there"
```

```
Hi           there
```

```
$ echo Hi           there
```

```
Hi there
```

```
$ date
```

```
Tue Sep 17 03:33:03 PM CEST 2024
```

```
$ date --iso-8601
```

```
2024-09-17
```

```
$ rev
```

```
Hello      <Ctrl-D> End-Of-File (EOF) token
```

```
olleH
```

stdin, stdout and stderr are usually shown together on the terminal

```
user@host:~$ ls *.txt doesNotExist.tar  
file.txt
```

```
ls: cannot access 'doesNotExist.tar': No such file or directory
```

stdin, stdout and stderr are usually shown together

```
user@host:~$ ls *.txt doesNotExist.tar  
file.txt
```

```
ls: cannot access 'doesNotExist.tar': No such file or directory
```

```
(shell prompt, stdout of shell)
```

stdin, stdout and stderr are usually shown together

```
user@host:~$ ls *.txt doesNotExist.tar  
file.txt
```

```
ls: cannot access 'doesNotExist.tar': No such file or directory
```

(shell prompt, stdout of shell)

(user input, stdin of shell)

stdin, stdout and stderr are usually shown together

```
user@host:~$ ls *.txt doesNotExist.tar  
file.txt
```

```
ls: cannot access 'doesNotExist.tar': No such file or directory
```

```
(shell prompt, stdout of shell)
```

```
(user input, stdin of shell)
```

```
(stdout of ls)
```

stdin, stdout and stderr are usually shown together

```
user@host:~$ ls *.txt doesNotExist.tar  
file.txt
```

```
ls: cannot access 'doesNotExist.tar': No such file or directory
```

```
(shell prompt, stdout of shell)
```

```
(user input, stdin of shell)
```

```
(stdout of ls)
```

```
(stderr of ls)
```

Shells allow to **redirect** stdin, stdout and stderr

`cmd < file` Read input from `file` instead of the keyboard

`cmd > file` Send output to `file` instead of the display

`cmd 2> file` Send **errors** to `file` instead of the display

`cmd &> file` Send **both** stdout and stderr to `file`

- Variants: `>>`, `2>>`, `&>>` append, otherwise **overwrite!!**
- Composable: `cmd < in_file >> out_file 2> err_file`
- `/dev/null`: special file, anything sent there is **discarded**

Example

```
$ ls -l foo Makefile
ls: cannot access 'foo': No such file or directory
-rw-r--r-- 1 florian users 2010 Sep 17 13:05 Makefile
```

```
$ ls -l foo Makefile 2>/dev/null
-rw-r--r-- 1 florian users 2010 Sep 17 13:05 Makefile
```

```
$ ls -l foo Makefile > lsMakefile.txt
ls: cannot access 'foo': No such file or directory
```

```
$ rev lsMakefile.txt
elifekaM 50:31 71 peS 0102 sresu nairolf 1 --r--r-wr-
```

(2010 is the filesize)

Example

```
# "lsMakefile.txt" is a command line argument to rev
$ rev lsMakefile.txt
elifekaM 50:31 71 peS 0102 sresu nairolf 1 --r--r-wr-
```

```
# "lsMakefile.txt" is piped into rev via stdin
$ rev < lsMakefile.txt
elifekaM 50:31 71 peS 0102 sresu nairolf 1 --r--r-wr-
```

```
# the output of these commands is exactly the same
$ rev Makefile | sha1sum ; rev < Makefile | sha1sum
890e182309cad29e44730e92dfb64de701f4320b  -
890e182309cad29e44730e92dfb64de701f4320b  -
```

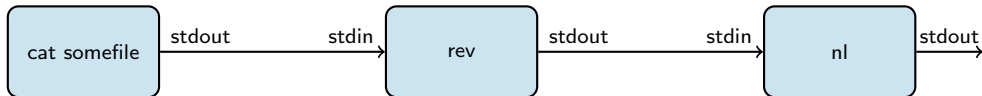
```
$ echo "Hello class" > somefile
$ cat somefile
Hello class
$ rev < somefile
ssalc olleH
```

```
$ echo "More text." >> somefile
$ cat somefile
Hello class
More text.
$ nl somefile
  1  Hello class
  2  More text.
```

Pipes connect processes

Essentially they redirect the stdout of one process to the stdin of another

```
$ cat somefile | rev | nl
  1  ssalc olleH
  2  .txet eroM
```



Doug McIlroy, 1978: (summarized)

- Write programs that do one thing and do it well. (DOTADIW)
- Write programs to work together.
- Write programs to handle text streams, because that is a universal interface.

Combine small, sharp tools and the use of a common underlying format (the line-oriented, plain text file) to accomplish larger tasks

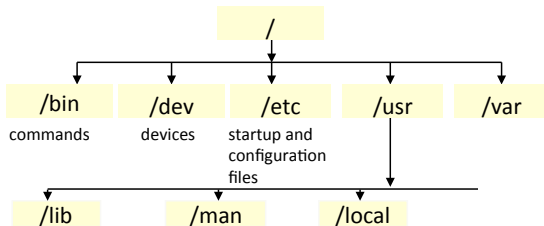
- Programs executed on the shell are **child processes** of the shell.
 - **<Ctrl+Z>** stops currently active job
 - `jobs` – status of processes started in the current shell
 - `fg %n` – continue job *n* in foreground
 - `bg %n` – continue job *n* in background
 - `'&'` at the end of a command starts it in the background

- Multiple commands:

Command sequence	Resulting behaviour
<code>cmd1 ; cmd2</code>	Execute commands subsequently
<code>cmd1 && cmd2</code>	Execute <code>cmd2</code> only if <code>cmd1</code> succeeds
<code>cmd1 cmd2</code>	Execute <code>cmd2</code> only if <code>cmd1</code> fails
<code>cmd1 & cmd2</code>	Start <code>cmd1</code> in background and <code>cmd2</code> in foreground
<code>(cmd1 ; cmd2)</code>	Execute both commands in a subshell

- Example: `$ (sleep 10; date) > outfile &`

- Hierarchical structure of files
- Each file belongs to one **user** and one **group**, and has a **path**
- Wide range of input/output resources are simple streams of bytes exposed through the filesystem name space → **“everything is a file”**
 - Documents, directories, symbolic links
 - Character-, block special files (devices; e.g. hard drives, keyboards, printers)
 - Named pipes
 - Sockets (e.g. TCP/IP sockets, UNIX domain sockets)



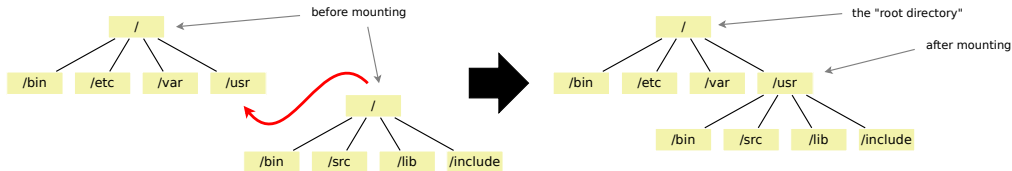
Standardised directory structure for UNIX-like operating systems

- / Primary hierarchy (root directory)
 - /bin: Essential command binaries (for all users)
 - /etc: Configuration files
 - /dev: Devices
 - /lib: Libraries essential for the binaries in /bin and /sbin
 - /home: Users' home directories
 - /media: Mount points for removable media
 - /mnt: Temporarily mounted file systems
 - /opt: Optional application software packages
 - /proc: Virtual filesystem providing process and kernel information as files
 - /sbin: Essential system binaries
 - /var: Variable files, whose content is expected to continually change during normal operation of the system (log files, spools, temporary e-mails)

Nested hierarchies:

- `/usr`: Secondary hierarchy for shareable, read-only data (contains the majority of multi-user utilities and applications)
 - `/usr/bin`
 - `/usr/etc`
 - ...
- `/usr/local`: Tertiary hierarchy for local data, specific to the host
 - `/usr/local/bin`
 - `/usr/local/etc`
 - ...

- All files and directories appear under the root directory, even if they are stored on different physical or virtual devices
- File system to be mounted is either:
 - locally available (hard-drive partitions, removable media)
 - a network resource (e.g. using NFS),
 - or contained in a file itself (e.g. loop device for ISO-Images)



cd: change directory

pwd: print working directory

ls: list directory contents

mkdir create directories

rmdir: delete directories

.: current directory

..: parent directory

```
$ cd ; pwd
/home/jdoe
$ mkdir test ; cd test ; pwd
/home/jdoe/test
$ echo "Hello class" > textfile
$ ls -l
-rw-rw-r-- 1 jdoe jdoe  12 Oct  6 11:15 textfile
$ cd .. ; pwd
/home/jdoe
```

Absolute paths: start with /, from the root directory

e.g. `$ cat /etc/passwd`

Relative paths: do **not** start with /, and are related to the current directory

e.g. `$ cat ../tmpfile`

- Pattern matching to express multiple filenames
 - * zero or more characters
 - ? a single character
 - [xyz] one of 'x', 'y' or 'z'
 - [a-i] one in the range from 'a' to 'i'
- The shell interprets wildcards and expands into lists of filenames
- This operation is known as [globbing](#).
- Use wildcards as normal characters by quoting or a preceding backslash (\)

Examples

```
$ ls
myfile  prog  prog.c  proG.c  t1  t2
t3      t4    test1   test1.c
```

*	<i>(all files listed above)</i>
t*	t1 t2 t3 t4 test test.c
t?	t1 t2 t3 t4
t[12]	t1 t2
pr*.c	proG.c prog.c
*[1-4].c	test1.c

- Stored as file attribute, shown with `ls -l`

$\underbrace{-}_{\text{special}} \quad \underbrace{4\ 2\ 1}_{\text{user}} \quad \underbrace{4\ 2\ 1}_{\text{group}} \quad \underbrace{4\ 2\ 1}_{\text{others}}$
 rwx rwx rwx

The permissions get added, r +4 +2 +1
7 to have the user do everything, and 4 to read only

- First character = type of file: normal (-), directory (d), socket (s), symbolic link (l), pipe (p), character special device (c), block special device (b)
- Other characters = permission to **r**ead/**w**rite/**e**xecute for user/group/others
- `chmod` – change file mode bits (= permissions)
 - with octal representation, e.g. `chmod 764 textfile`
 - textual specification, e.g. `chmod ugo+x,g-w textfile`
- `chown` – change owner of file
- Only owner or root can change permissions/owner

Example

```
$ touch regularFile      # create empty file
$ nano script.sh         # open a text editor, create file
$ chmod +x script.sh    # add execute permissions for user
$ ls -l regularFile script.sh
-rw-r--r-- 1 florian users  0 Sep 17 15:52 regularFile
-rwxr-xr-x 1 florian users 46 Sep 17 15:53 script.sh
$ ./script.sh            # run the script
Hello World
$ ./regularFile          # can't run a regular file
bash: ./regularFile: Permission denied
$ chmod 744 script.sh    # set new permissions
$ ls -l regularFile script.sh
-rw-r--r-- 1 florian users  0 Sep 17 15:52 regularFile
-rwxr--r-- 1 florian users 46 Sep 17 15:53 script.sh
# now only the user/owner can execute the script
```

- Everything is a string
- Created at first assignment

```
$ FILE=/tmp/dummy.txt
```

- Usage:

```
$ ls /tmp  
dummy.txt dummy.txt.bak  
$ rm $FILE ${FILE}.bak
```

- Export to environment for subsequently started processes:

```
$ export FILE
```

combined with assignment:

```
$ export FILE=/tmp/dummy.txt
```

- \$HOME ... Home directory
- \$USER ... User name
- \$? ... Exit status of the last command
- \$PATH ... Program path

```
$ echo $PATH
/usr/local/bin:/usr/bin:/usr/local/sbin:/usr/sbin
```
- If you want to execute a program in the current directory:

```
$ ./myprogram
```
- Print environment variables with `env`

Examples

- File management

ls	list directory contents
cd	change the working directory
pwd	print filename of working directory
cp, mv	move (rename) files
ln	make links to files
mkdir	make directories
rm, rmdir	remove files and directories
chmod, chown	change file mode bits, owner
du	estimate file space usage
file	determine file type

- Process management

jobs	display status of jobs in current shell session
fg, bg	run job in foreground/background
ps, pstree	snapshot of current processes/process hierarchy
kill	send a signal to a process

Examples

- Text processing

cat	concatenate files to standard output
sort	sort lines of text files
nl	number lines of files
wc	print line, word, and byte counts
cut	remove sections from each line
tr	translate or delete characters
tac	concatenate and print files in reverse
rev	reverse lines of a file
grep	print lines matching a pattern
sed	stream editor for filtering and transforming text

Examples

- Utilities

echo	print arguments to stdout
more, less	pager
date, cal	print current time and time/calendar
tar	archiving utility
make	build utility
ssh	SSH client (remote login program)
gcc	GNU compiler collection C compiler

- Editors vim, emacs

- ... and many many more

- see `$ man command` for more information

- How can processes interact?
 - **stream** of data (pipes, stream sockets)
 - sending **messages** (message queues, datagram sockets)
 - accessing a **shared resource** (file, memory)
- Classification
 - **related vs. unrelated** processes
unrelated processes require named resources (system-wide namespace)
 - **implicit vs. explicit** synchronization
ensure orderly execution and access to a shared resource

What you will learn in this course

We assume. . .

You already know how to program in an imperative programming language.
This is not an introduction to programming!

Educational objectives

- How to write and compile a C program, use options and arguments, basic stream I/O
- Collaboration of unrelated processes through shared memory and synchronization with semaphores
- How to create child processes, communicate through unnamed pipes
- Communicate through stream sockets